PCBAgent: An Agent-based Framework for High-Density Printed Circuit Board Placement

Lin Chen[1,2], Ran Chen[2], Shoubo Hu[2], Xufeng Yao[2,3], Zhentao Tang[2], Shixiong Kai[2], Siyuan Xu[2], Mingxuan Yuan[2], Jianye Hao[2], Bei Yu[3], Jiang Xu[4] *

ABSTRACT

Recently, printed circuit board (PCB) placement has emerged as a significant challenge since the scale of PCB designs has rapidly enlarged. Furthermore, the presence of various types of constraints with differing tolerance priorities hampers the automation of PCB layout design, necessitating substantial manual effort. To address this problem, we introduce a novel agent-based framework that automatically generates PCB layouts meeting industrial constraints through user interactions. This framework includes two main agents: a reinforcement learning (RL)-based agent for layout inference and fine-tuning, and a large language model (LLM)-based agent for interactive optimization. Experimental results on 17 industrial tasks show that our framework outperforms other state-of-the-art methods.

ACM Reference Format:

Lin Chen[1,2], Ran Chen[2], Shoubo Hu[2], Xufeng Yao[2,3], Zhentao Tang[2], Shixiong Kai[2],, Siyuan Xu[2], Mingxuan Yuan[2], Jianye Hao[2], Bei Yu[3], Jiang Xu[4] . 2025. PCBAgent: An Agent-based Framework for High-Density Printed Circuit Board Placement. In *30th Asia and South Pacific Design Automation Conference (ASPDAC '25), January 20–23, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3658617.3697736

1 INTRODUCTION

With the growing demand for higher integration of electronic devices, PCB design density is continuously rising, while also requiring satisfying numerous design constraints. Consequently, finding an optimal solution for a PCB placement task is often time-consuming, leading to extended PCB design cycles and significant human effort.

Recent advancements in Integrated Chip (IC) placement [1–5] formulate the task as a Markov decision process, placing components sequentially and using RL algorithms [6]. Despite conceptual similarities with IC placement, PCB placement faces additional challenges like high density, irregular boundaries, and alignment requirements. These challenges lead to under-performance or failure of IC placement solutions in PCB tasks. Moreover, previous studies have primarily focused on optimizing HPWL (half perimeter wirelength) while neglecting other important factors, such as net intersection.

*[1]The Hong Kong University of Science and Technology, [2]Huawei Noah's Ark Lab, [3]The Chinese University of Hong Kong, [4]The Hong Kong University of Science and Technology (Guangzhou), corresponding author.

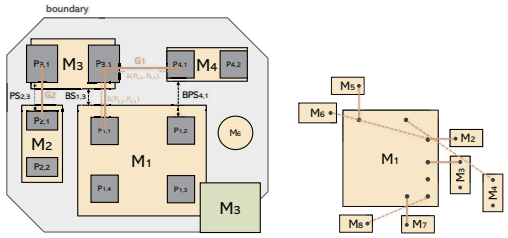
This work was done during the internship at Huawei Noah's Ark Lab and was supported by the Guangzhou-HKUST(GZ) Joint Funding Program (No. 2023A03J0013).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASPDAC '25, January 20–23, 2025, Tokyo, Japan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0635-6/25/01
https://doi.org/10.1145/3658617.3697736



(a) Fundamental notations. (b) Surface layer wire.

Figure 1: PCB placement examples.

Existing PCB placement approaches can be classified into two main categories. One class utilizes derivative-free algorithms, such as genetic algorithms [7] and particle swarm optimization [8], to develop meta-heuristics for searching valid placements [9–12]. The other class uses gradient-based methods. For instance, [13] solves the problem approximately using first-order methods within an alternating minimization framework. Two major limitations of these methods are: 1) inability to leverage past experience; and 2) inability to incremental optimization using expert evaluations. While some studies have pre-trained a model [5] to learn from expert results, the high variances in both human preferences and PCB tasks make it challenging to train a model with strong generalization capabilities.

To address these challenges, we propose a novel agent-based framework (PCBAgent) consisting of an RL agent and an LLM agent to learn knowledge from past experiences and human preferences. By incorporating trade-off processes encountered by engineers into an automated agent, we eliminate the requirement for hyperparameter tuning or pre-training. Our main contributions are as follows:

- We propose an RL agent that can provide legal placement results for high-density PCB tasks while enhancing their routability.
- We propose an LLM agent that can facilitate user interaction via natural language and achieve multi-round optimization.
- We first introduce a new metric called *surface layer wire* as a supplementary measure to HPWL.
- We demonstrate the superiority of our framework over existing algorithms on industrial PCB tasks.

2 PRELIMINARIES

2.1 PCB Placement Constraints

The PCB placement problem can be defined as a constrained optimization problem, with the objective of arranging necessary electronic components within a limited region on the PCB while adhering to various constraints. All constraints can be classified into: 1) *hard constraints*, which determine the legality of a placement result; 2) *soft constraints*, which assess the quality of a placement result.

Notations: Let M_t denote t-th electronic component to be placed, $P_{t,i}$ denote the i-th pad of M_t[1]. Then $\mathbb{M} = \{M_1, \ldots, M_n\}$, where n

[1]A pad is a metal area serving as a connection point for electronic components

represents the total number of components in a task, is the set containing the information of all components and their pads. Since the boundaries of the placement region can vary and take on irregular shapes in PCB scenarios, $\mathbb{B}$ denotes the feasible region enclosed by these irregular boundaries. Additionally, let $\delta(\cdot,\cdot)$ be a function that measures the spacing between two objects. Let G denote a net and $\mathbb{G} = \{G_1, \ldots, G_m\}$ denote the set of all nets in the placement task. Consequently, the result of a PCB placement task includes the coordinates $(\boldsymbol{x}, \boldsymbol{y})$ and rotation angles $(\boldsymbol{d})$ of all components $M_t \in \mathbb{M}$ inside the feasible region $\mathbb{B}$, while meeting all spacing requirements.

Hard Constraints. We involve three hard constraints in our placement problem. 1) *Overlapping constraint*: components should not have overlapping regions. 2) *Boundary constraint*: components should be within the feasible region, i.e. $M_t \subseteq \mathbb{B}, \forall M_t \in \mathbb{M}$. 3) *Spacing constraint*: the spacing between two objects (including both components' body and pads) should be greater or equal to a certain threshold $h(\cdot,\cdot)$, i.e., $\delta(o_t, o_{t'}) \geq h_{o_t,o_{t'}}, \forall o_t, o_{t'} \in \mathbb{M}$.

Violation of any hard constraint would result in an illegal placement result. Figure 1(a) shows an example of illegal result, in which M_3 is beyond the boundary and overlapped with component M_1.

Soft Constraints. One of the most crucial soft constraints is wirelength minimization, which is a commonly used metric for evaluating the routability of placement results. We adopt half perimeter wirelength (HPWL) to approximately estimate wirelength to reduce the computational cost [14]. In the example in Figure 1(a), the HPWL of net G_1 can be efficiently calculated by $\delta(P_{3,1}, P_{4,1}) + \delta(P_{3,1}, P_{1,1})$.

In addition to wirelength, PCB placement has its unique requirement of maximizing the number of surface layer wires.

Definition 1 (**Surface Layer Wire (SLW)**). *Let $e_{ti,t'j}$ be an electronically connected wire between two pads $P_{t,i}$ of M_t and $P_{t',j}$ of $M_{t'}$. $e_{ti,t'j}$ is called a surface layer wire if and only if:*

(1) its directions of fan-out from M_t and $M_{t'}$ are towards one of the two edges closest to $P_{t,i}$ and $P_{t',j}$, respectively (e.g., $e_{1,6}$ should fan out from the top or left edges of M_1));
(2) it does not intersect with components besides M_t and $M_{t'}$;
(3) $P_{t,i}$ and $P_{t',j}$ should have overlapping regions in either their horizontal or vertical projections.

We provide an illustration of SLW in Figure 1(b), where pads are represented as simplified dots for clarity. The solid lines are considered SLWs as they satisfy conditions (1)-(3). However, $e_{1,6}$, $e_{1,4}$, and $e_{1,8}$ violate conditions (1), (2), and (3), respectively[2]. Increasing the number of SLW (NSLW) can reduce PCB layers and improve routing quality, particularly for important components such as ICs.

2.2 Problem Formulation

Considering all the constraints mentioned above, our placement optimization problem can be formulated as

$$\begin{aligned} \min_{\boldsymbol{x},\boldsymbol{y},\boldsymbol{d}} \quad & \text{HPWL}(\boldsymbol{x},\boldsymbol{y},\boldsymbol{d},\mathbb{M},\mathbb{G}) + \lambda \cdot 1/\text{NSLW}(\boldsymbol{x},\boldsymbol{y},\boldsymbol{d},\mathbb{M},\mathbb{G}), \\ s.t. \quad & \text{Overlap}(\boldsymbol{x},\boldsymbol{y},\boldsymbol{d},\mathbb{M}) = 0, \\ & \text{BeyondBoundary}(\boldsymbol{x},\boldsymbol{y},\boldsymbol{d},\mathbb{M},\mathbb{B}) = 0, \\ & \text{InsufficientSpacing}(\boldsymbol{x},\boldsymbol{y},\boldsymbol{d},\mathbb{M}) = 0, \end{aligned} \tag{1}$$

where $\boldsymbol{x} = [x_1, x_2, \ldots, x_n]^T$ denotes the horizontal coordinates of components, $\boldsymbol{y}$ and $\boldsymbol{d}$ similarly denote the vertical coordinates and rotation angles. Functions HPWL$(\cdot)$, NSLW$(\cdot)$, BeyondBoundary$(\cdot)$, Overlap$(\cdot)$, and InsufficientSpacing$(\cdot)$ are used to calculate the HPWL, the NSLW, the number of components beyond the boundary, the number of overlapping components, and the number of components violating spacing constraint, respectively. λ is the combined weight.

2.3 Challenges in PCB Placement

PCB placement exhibits unique challenges compared to IC placement due to its distinct characteristics. 1) **High density**: Despite the component scale of PCBs generally being smaller than that of chips, PCBs have a much higher density (up to 76% in Table 1), complicating the satisfaction of all hard constraints. 2) **Irregular boundaries and spacings**: PCBs feature irregular boundaries and varying spacing requirements, posing specific challenges that are overlooked by IC placers and previous PCB studies. 3) **Surface layer wire optimization**: Balancing NSLW maximization with HPWL minimization is challenging, yet crucial for achieving satisfactory PCB placement results. This aspect, vital in industry, has been neglected in academic research. 4) **Rotation feasibility**: Most machine learning-based placers for chips only generate $(\boldsymbol{x}, \boldsymbol{y})$ coordinates as the placement results, either setting the rotation degree $(\boldsymbol{d})$ to 0 by default or pre-determining the rotation for each component [4, 5]. However, the rotation degree is crucial for high-density PCBs. This additional variable ranging from 0 to 360 further complicates the PCB placement problem. 5) **Placement sequence**: The sequence significantly affects the efficiency of RL-based methods, formulated as a Markov decision process. This factor is mostly ignored in previous studies despite its importance [4, 5].

3 ARCHITECTURE OVERVIEW

Figure 2 displays our proposed PCBAgent architecture with the workflow outlined on the left. The RL agent processes a PCB placement task and automatically generates a result based on task constraints. The LLM agent then analyzes this result and translates user descriptions into commands for the RL agent. After multiple interactions among the RL agent, LLM agent, and users, the PCBAgent produces an optimized placement that improves HPWL and NSLW while aligning with user intentions. Further design details of the RL and LLM agents are discussed in Sections 4 and 5, respectively.

4 AGENT FOR PLACEMENT

Our RL agent, depicted in the middle of Figure 2, utilizes decision transformer (DT) [15] as the core of its policy π_θ, which predicts action probabilities. At each decision step t for placing component M_t, the RL agent inputs previous states $\boldsymbol{s}_{:t}$, rewards $\boldsymbol{r}_{:t}$, actions $\boldsymbol{a}_{:t-1}$, and hindsight information (HI) and outputs an action a_t. Here, $\boldsymbol{s}_{:t}$, $\boldsymbol{r}_{:t}$, and $\boldsymbol{a}_{:t-1}$ represent all respective values up to timestep t or $t-1$, and a_t specifies the placement coordinates and rotation (x_t, y_t, d_t) of M_t. HI, following [5], is a learned embedding representing the task's statistical properties. When the final component M_T is placed, return-to-gos are calculated as $g_T = \sum_{t'=t}^{T} r_{t'}$, and the entire trajectory $\tau = (g_1, s_1, a_1, g_2, s_2, a_2, ..., g_T, s_T, a_T)$ is added to the replay buffer. The model is fine-tuned using prioritized experience replay [16].

[2]Since each component has only one wire to M_1, pads' indexes are omitted for readability

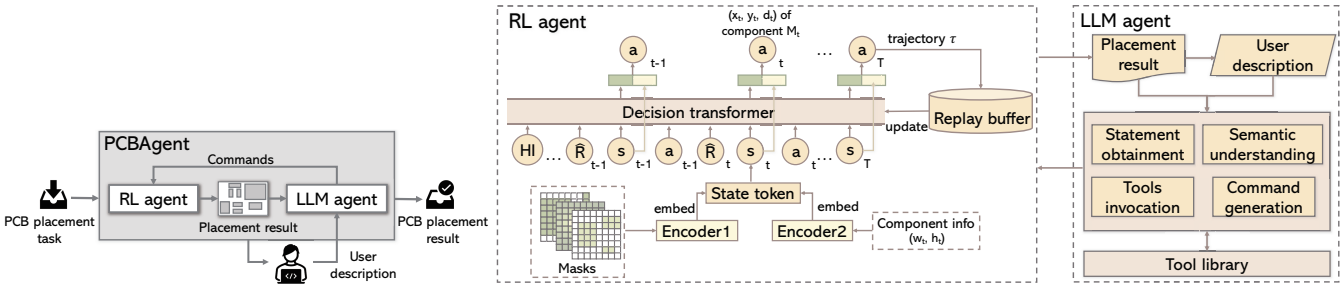
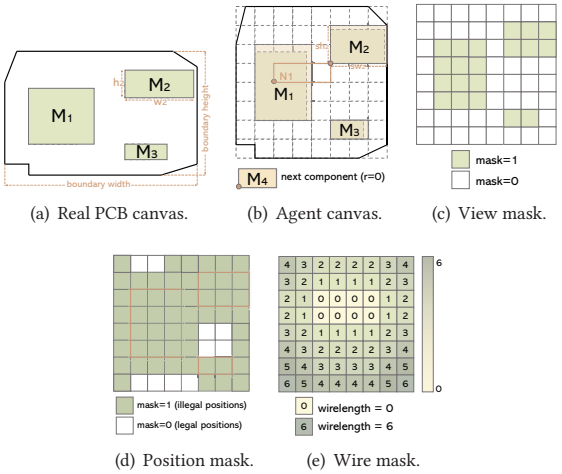


Figure 2: Architecture overview of PCBAgent.



(a) Real PCB canvas. (b) Agent canvas. (c) View mask.

(d) Position mask. (e) Wire mask.

Figure 3: An example for placement masks (size = 8 x 8).

4.1 State Token

State token is essential in RL-based placement, representing the cumulative status at each step. We adopt mask definitions from [4] to construct the state token and carefully redesign them to fit our PCB scenario as shown in Figure 3.

Canvas Scaling. A real PCB canvas should first be converted into a size appropriate for the RL agent, as shown in Figures 3(a) and 3(b). The irregular and non-square boundary is scaled to a square canvas of size $N \times N$. For a component with width w and height h, its scaled size is computed as $\tilde{w} = \left\lceil \frac{w \times N}{W} \right\rceil$, $\tilde{h} = \left\lceil \frac{h \times N}{H} \right\rceil$, where W, H represent the original width and height of the boundary, respectively, and $\lceil \cdot \rceil$ denotes the ceiling function for rounding up.

View Mask. The view mask $\boldsymbol{s}^v \in \{0,1\}^{N \times N}$ is a binary matrix containing 0 and 1 values, where a grid cell with a value of 1 indicates that a component has occupied this cell. As displayed in Figure 3(c), the view mask provides an overview of the current canvas.

Position Mask. The position mask $\boldsymbol{s}^p \in \{0,1\}^{N \times N}$ also only contains 0 and 1 values. In $\boldsymbol{s}^p$, a grid cell with a value of 0 indicates a legal position, which means that placing the next component in that position would satisfy all the hard constraints. Conversely, a grid cell with a value of 1 represents an illegal position. As displayed in Figure 3(d), the position mask distinguishes whether a position is legal for the next component or not.

Wire Mask. The wire mask $\boldsymbol{s}^w \in \mathbb{R}^{N \times N}$ is a real matrix that represents the increase of HPWL values for the next component. As shown in Figure 3(e), a grid cell with a value of 2 indicates that placing the next component in that position would result in a 2-unit increase of HPWL. The wire mask enables our RL agent to identify grids with the minimum increase of HPWL thus guiding it towards results with better routability and significantly reducing the vast search space of the PCB placement task.

State Token Construction. Given different rotation angles of the next component to be placed, position mask $\boldsymbol{s}^p$ and wire mask $\boldsymbol{s}^w$ would also be different. In PCBAgent, 4 possible rotation angles (i.e., 0, 90°, 180°, 270°) are considered for each component and Figure 3 illustrates an example when the rotation is 0. As a results, there would be 4 position masks $\boldsymbol{s}^{p_0}, \ldots, \boldsymbol{s}^{p_3}$ and 4 wire masks $\boldsymbol{s}^{w_0}, \ldots, \boldsymbol{s}^{w_3}$ correspond to each component. Then the state token is obtained by concatenating the encoded embedding from all three types of masks with the encoded embedding of the next component's dimension.

4.2 Legalization for Hard Constraints

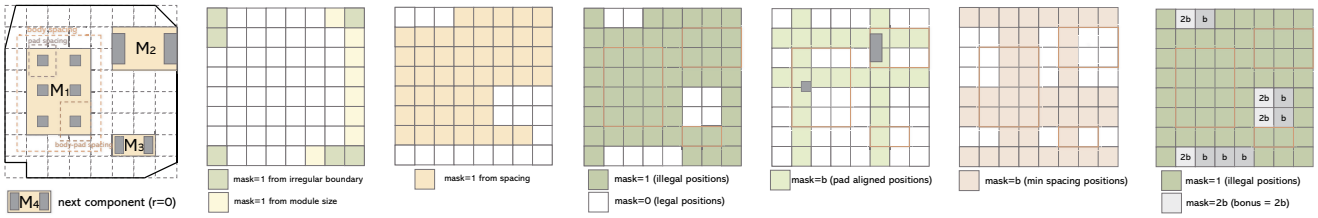
We achieve hard constraints using delicately designed position masks that integrate boundary and spacing information. The boundary mask (shown in Figure 4(b)) marks the grids within the boundary. Specifically, any grid that intersects with the irregular boundary lines is marked 1, indicating that it is an illegal position for placing the next component. The spacing mask (shown in Figure 4(c)) marks the grids satisfying spacing requirements for the next component. In this case, the spacing thresholds are set as follows: $h(M_1, M_4) = h(M_2, M_4) = 1$ grid length, and $h(M_3, M_4) = 0$. Grids that would induce insufficient spacing between the next component with previously placed components are marked 1. Since $h(\cdot, \cdot)$ is non-negative, satisfying the spacing constraint ensures no overlap in the placement result.

Finally, the position mask for legalization $\boldsymbol{s}^p$ is derived by taking the logical sum of the boundary mask and spacing mask as shown in Figure 4(d). Then the position mask is further adopted to constrain the action sampling space. The RL agent's output logits for each step are adjusted by subtracting the product of the position mask and a large positive value, significantly reducing the probability of sampling actions with illegal positions.

4.3 Soft Constraints Optimization

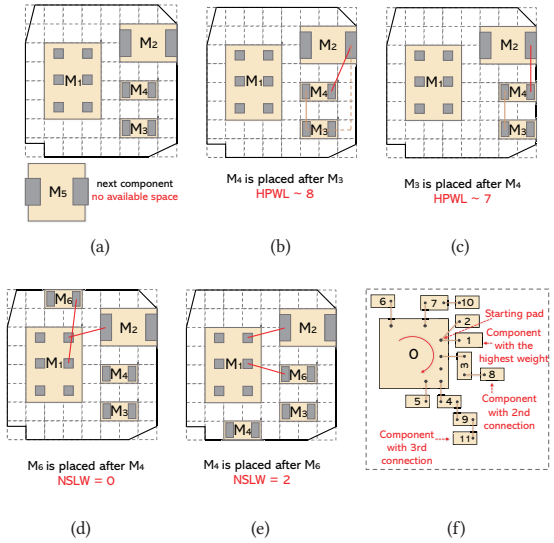
Except for the online fine-tuning mechanism of the DT algorithm to optimize the placement policy (as illustrated later), we also propose simple yet efficient offline techniques to enhance the soft constraints.

Placement Sequence Optimization. An improper placement sequence for the RL agent can lead to a poor-quality layout that fails to satisfy both the hard and soft constraints. Figure 5 demonstrates the impact of sequence on layout quality. For instance, placing a large component (M_5) later might leave insufficient space due to earlier non-compact placements, as illustrated in Figure 5(a). Altering the



(a) Agent canvas. (b) Boundary mask. (c) Spacing mask. (d) Position mask. (e) Pad alignment. (f) Spacing minimization. (g) Final mask.

Figure 4: Optimized position mask formulation.



(a) (b) (c) (d) (e) (f)

Figure 5: Placement sequence examples.

order of M_3 and M_4 leads to varying HPWLs, shown in Figures 5(b) and 5(c), with Figure 5(c) achieving a lower HPWL because placing M_4 first prioritizes its connection to M_2 over M_3. Different sequences also affect NSLW, as depicted in Figures 5(d) and 5(e).

Recognizing the critical role of placement sequence, we optimize it by adopting strategies used by PCB design experts. Unlike IC placement, which relies primarily on net information, our approach is chip-centered. We prioritize placing chips first and then order their pads in a clockwise or counterclockwise fashion. The placement of other components is then organized based on their connection to the chip pads. Each component is assigned a weight based on a linear combination of its size, number of connected nets, and its connection to the chip's pad round and level. Generally, we expect that the component directly connected to the pads belonging to the outermost round of the chip, with a larger size and more connected nets, will be assigned a higher weight.

Figure 5(f) demonstrates our sequencing method. Initially, the chip's order is set to zero. We then select the pad linked to the highest-weight component as the starting pad. Components directly connected to the chip are placed clockwise. Following that, we determine the order of components with second connections based on their weights, and then proceed to decide on the third connection.

Pad Alignment and Spacing Minimization. Alignment is an important requirement in PCB placement as it directly affects NSLW and ensures visually pleasing layouts. Therefore, we utilize a simple technique by assigning additional bonuses to aligned positions in the position mask. As depicted in Figure 4(e), there are two existing pads connected to the next component M_4. The grid cells where the pad of M_4 aligns with the connected pads are assigned b values.

Similarly, we award bonuses to positions that meet the minimum spacing requirements. As shown in Figure 4(f), a grid cell marked with a value of b indicates that placing M_4 there would satisfy the minimum spacing criteria with one of the existing components.

Figure 4(g) displays the final position mask, merging the original position mask with the pad alignment and spacing minimization masks. Only legal positions receive non-zero bonuses. Typically, we set b to be a small negative value, such as -0.01. By incorporating these bonuses into the position mask, the probability of selecting positions that are aligned and ensure minimum spacing increases.

HPWL-guided Action Search. In addition to legalizing action logits with the position mask, we use the wire mask to guide our RL agent towards results with lower HPWL. We establish a maximum HPWL threshold for candidate action logits, adjusting those exceeding the threshold by subtracting a large positive value. Consequently, this adjustment increases the likelihood of sampling actions that result in shorter HPWL.

Algorithm 1 Trajectory Generation (RL Agent Inference)

Input: task information $\mathbb{T}$, policy π_θ.
Output: PCB placement trajectory $\tau = (\boldsymbol{s}, \boldsymbol{g}, \boldsymbol{a}, score)$.

1: Compute the placement sequence seq (Section 4.3);
2: **for** M_t in seq **do**
3: Construct the state token $\boldsymbol{s}_t$ (Section 4.1);
4: Compute the increase of HPWL as the reward signal $r_t = \text{HPWL}_{t-1} - \text{HPWL}_{t-2}$;
5: Construct τ_t with $\boldsymbol{s}_{:t}$, $\boldsymbol{g}_{:t}$ and $\boldsymbol{a}_{:t-1}$;
6: Sample action a_t from current action distribution $\pi_\theta(a|\tau_t)$;
7: **end for**
8: Compute the score of current trajectory with Equation (2);

4.4 Placement with RL Agent

Our proposed RL agent conducts a placement task in two stages: 1) trajectory generation; and 2) RL agent online fine-tuning.

Trajectory Generation. Algorithm 1 illustrates the process of how the RL agent generates one trajectory τ. Firstly, the task information $\mathbb{T} = \{\mathbb{M}, \mathbb{G}, \mathbb{B}\}$ which includes component, net, and boundary information, is provided as input to the RL agent. The placement sequence is then calculated based on the logic described in Section 4.3 (line 1). Following that sequence, the RL agent proceeds to place components step-by-step using the DT inference mechanism (lines 2-7). At step t, the action a_t is determined based on the cumulative tokens τ_t, which includes the state tokens up to step t ($\boldsymbol{s}_{:t}$), the

Algorithm 2 RL Agent Online Fine-tuning

Input: task information $\mathbb{T}$, policy π_θ, replay buffer $\mathcal{B}$.
Output: PCB placement result $(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{d})$.

1: **for** i in n_{iters} **do**
2: $n_\tau = 0$;
3: **while** $n_\tau < N_\tau$ **do**
4: τ =Trajectory_Generation($\mathbb{T}$, π_θ) (Algorithm 1);
5: **if** hard constraints are satisfied **then**
6: Add τ to replay buffer $\mathcal{B}$;
7: $n_\tau \leftarrow n_\tau + 1$;
8: **end if**
9: **end while**
10: Adopt prioritized replay to sample a trajectory set from $\mathcal{B}$;
11: Update π_θ using current trajectory set following Equation (3);
12: **end for**
13: τ_{out} =Trajectory_Generation($\mathbb{T}$, π_θ);

return-to-go tokens up to step t ($\boldsymbol{g}_{:t}$), and the previous action tokens up to step $t-1$ ($\boldsymbol{a}_{:t}$). Here, g_t is calculated based on the reward signal $r_t = HPWL_{t-1} - HPWL_{t-2}$. After all components are placed, an overall evaluation (line 8) of the placement result is calculated as

$$score = \lambda_1 \cdot 1/\text{HPWL}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{d}) + \lambda_2 \cdot \text{NSLW}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{d}), \quad (2)$$

where a higher score signifies better fulfillment of soft constraints.

Online Fine-tuning. Algorithm 2 illustrates the online fine-tuning algorithm of our RL agent. The objective of fine-tuning is:

$$\min_\theta \mathcal{L}(\theta) := -\mathbb{E}_{\mathcal{B}(\tau)}[\omega(\tau) \log \pi_\theta(a_t|\tau_t)], \quad (3)$$

where $\pi_\theta(a_t|\tau_t)$ is the probability of action a_t, $\mathbb{E}_{\mathcal{B}(\tau)}$ is the expectation over replay buffer $\mathcal{B}$ containing τ. $\mathcal{B}$ is implemented as a priority queue, where the prioritization weight $\omega(\tau)$ is calculated as

$$\omega(\tau) = \frac{\exp(score(\tau)/\alpha)}{\mathbb{E}_{\mathcal{B}(\tau)}[\exp(score(\tau)/\alpha)]}, \quad (4)$$

where α denotes the sampling temperature. Only trajectories that satisfy the hard constraints are inserted into $\mathcal{B}$ (lines 5-8). After n_{iters} iterations of fine-tuning, a final trajectory τ_{out} is generated using the updated policy π_θ. The actions $\boldsymbol{a}_{out}$ in τ_{out} are converted into the output placement result $(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{d})$ (lines 13).

5 AGENT FOR INTERACTIVE OPTIMIZATION

Due to the complex trade-offs among soft constraints in PCB placement, automatic algorithms often struggle to meet all user requirements. Addressing this typically requires extensive tuning of the algorithm's hyper-parameters. Therefore, we propose an LLM agent that incrementally optimizes previous placement results based on user intentions.

LLM Agent Overview. The right part of Figure 2 outlines our LLM agent, featuring four key components: statement obtainment, semantic understanding, tool invocation, and command generation. Statement obtainment captures current placement outcomes and user inputs, which are then translated into programming representations through semantic understanding. Tools from the tool library are subsequently invoked to generate commands for the RL agent.

Intention-guided Incremental Optimization. We utilize an in-context learning approach that integrates task demonstrations with a focus on defect descriptions in placement tasks. The tool library serves as a resource for analyzing PCB placement outcomes. The LLM agent employs one or more tools to evaluate the results and identify components deviating from constraints. The RL agent then adjusts the scoring in Equation (2) based on user intentions and modifies trajectory weights in the replay buffer. To avoid drastic changes, we only rearrange these violating components following Algorithm 2. By continuously interacting with users through the LLM agent and fine-tuning via the RL agent, our PCBAgent progressively optimizes placement results in alignment with user feedback.

Table 1: PCB Benchmark statistics.

Designs	W(mil)xH(mil)	Density	$\lvert\mathbb{M}\rvert$	$\lvert\mathbb{M}_{chip}\rvert$	$\lvert\mathbb{P}\rvert$	$\lvert\mathbb{G}\rvert$
Task1	206x114	0.57	18	2	56	18
Task2	147x96	0.48	19	1	48	16
Task3	337x192	0.57	21	1	68	17
Task4	229x201	0.61	24	1	70	17
Task5	421x237	0.56	27	1	82	23
Task6	256x295	0.56	29	1	84	28
Task7	426x205	0.71	33	2	298	41
Task8	301x482	0.56	34	2	140	40
Task9	454x410	0.71	36	1	213	62
Task10	380x220	0.52	42	1	150	43
Task11	465x315	0.58	42	5	117	31
Task12	346x256	0.76	56	1	329	85
Task13	498x494	0.51	68	7	261	60
Task14	434x499	0.45	83	3	280	67
Task15	807x682	0.33	177	6	517	70
Task16	1251x941	0.52	304	13	1036	196
Task17	1137x1159	0.40	414	17	1371	310

6 EXPERIMENTAL RESULTS AND ANALYSIS

6.1 Benchmarks and Settings

PCB Benchmarks. We evaluated our method on 17 real PCB tasks from the industry, detailed in Table 1. Here, $W \times H$ represents the maximum width and height of the PCB boundary. Density is computed as the sum of component areas divided by the available area within the boundary. $|\mathbb{M}|$, $|\mathbb{M}_{chip}|$, $|\mathbb{P}|$, and $|\mathbb{G}|$ denote the numbers of components, chips, pads, and nets, respectively. These benchmarks cover various PCB characteristics, such as boundary shapes, densities, and component types, providing more diversity than those in [13, 17]. Notably, the average placement density is 0.55 (0.47 in [13]), peaking at 0.76 in Task12. A density level of around 70% is notably high for PCB benchmarks, necessitating adequate spacing allocation. This high density underscores the challenges in PCB placement and highlights the importance of our approach.

Parameters. Our model architecture follows the major setup of GPT in [5]. λ_1 and λ_2 in Equation (2) are set to $(1000, 0.01)$. The grid, replay buffer, and fine-tuning batch sizes are set to $(128, 64, 8)$.

Baseline Methods. We evaluate our PCBAgent against advanced RL-based methods ChipFormer [5] and MaskPlace [4], as well as a mixed-integer linear program-based method (MILP) [18], using their default settings. MILP is a post-layout optimization method using an initial layout as input. So we feed MILP with direct inference results from PCBAgent and compare the optimized layouts to those from PCBAgent's optimization stages. Additionally, we compare our placement results with those manually placed by expert engineers.

6.2 Main Placement Results

Table 2 compares HPWL, NSLW, and legal states among baseline methods, manual layouts, and our PCBAgent. Legal states are represented as a 3-element tuple indicating compliance with within-boundary, non-overlapping, and sufficient spacing constraints (1

Table 2: Main results on PCB Design benchmark.

	Manual			MaskPlace			ChipFormer			MILP			Ours		
	HPWL	NSLW	LegalStates	HPWL	NSLW	LegalStates	HPWL	NSLW	LegalStates	HPWL	NSLW	LegalStates	HPWL	NSLW	LegalStates
Task1	339.31	15	(1, 1, 1)	-	-	(0, 0, 0)	-	-	(1, 0, 0)	657.85	13	(0, 1, 0)	439.69	12	(1, 1, 1)
Task2	331.94	16	(1, 1, 1)	389.65	9	(1, 1, 0)	-	-	(1, 0, 0)	882.49	14	(0, 1, 0)	331.33	15	(1, 1, 1)
Task3	1405.91	18	(1, 1, 1)	917.43	8	(1, 1, 0)	1136.63	12	(0, 1, 0)	2079.44	8	(0, 1, 0)	1138.34	20	(1, 1, 1)
Task4	1057.11	16	(1, 1, 1)	2299.46	9	(0, 1, 0)	-	-	(1, 0, 0)	1861.28	14	(0, 1, 0)	1257.96	17	(1, 1, 1)
Task5	2293.49	19	(1, 1, 1)	1286.0	17	(0, 1, 0)	1311.26	16	(0, 1, 0)	3846.8	10	(0, 1, 0)	1472.74	19	(1, 1, 1)
Task6	2553.41	21	(1, 1, 1)	3509.41	7	(0, 1, 0)	2079.01	11	(0, 1, 0)	6717.16	7	(0, 1, 0)	2207.97	21	(1, 1, 1)
Task7	1859.88	21	(1, 1, 1)	-	-	(0, 0, 0)	-	-	(0, 0, 0)	2544.58	10	(0, 1, 0)	4835.38	11	(1, 1, 1)
Task8	3067.47	21	(1, 1, 1)	4310.4	13	(0, 1, 0)	-	-	(0, 0, 0)	4429.01	11	(0, 1, 0)	2424.28	26	(1, 1, 1)
Task9	4660.42	30	(1, 1, 1)	3935.82	15	(0, 1, 0)	4084.23	13	(0, 1, 0)	12293.97	8	(0, 1, 0)	5885.84	23	(1, 1, 1)
Task10	1662.89	33	(1, 1, 1)	2738.96	13	(0, 1, 0)	-	-	(0, 0, 0)	3853.03	15	(0, 1, 0)	1204.6	32	(1, 1, 1)
Task11	1617.68	33	(1, 1, 1)	2521.63	21	(0, 1, 0)	2381.1	21	(0, 1, 0)	3398.46	27	(0, 1, 0)	2443.96	29	(1, 1, 1)
Task12	2510.05	45	(1, 1, 1)	1887.75	22	(0, 1, 0)	2169.45	21	(0, 1, 0)	-	-	(0, 0, 0)	1973.74	39	(1, 1, 1)
Task13	5217.38	64	(1, 1, 1)	-	-	(0, 0, 0)	5661.01	41	(1, 1, 0)	13821.4	24	(0, 1, 0)	4590.76	58	(1, 1, 1)
Task14	9876.12	41	(1, 1, 1)	7068.71	28	(0, 1, 0)	-	-	(1, 0, 0)	-	-	(1, 0, 0)	6908.99	46	(1, 1, 1)
Task15	11642.59	46	(1, 1, 1)	5711.02	34	(1, 1, 0)	5537.33	33	(1, 1, 0)	-	-	(1, 0, 0)	5664.45	46	(1, 1, 1)
Task16	41428.99	184	(1, 1, 1)	-	-	(0, 0, 0)	96211.17	94	(0, 1, 0)	-	-	(0, 0, 0)	31792.29	207	(1, 1, 1)
Task17	46976.58	297	(1, 1, 1)	61319.08	157	(0, 1, 0)	189872.46	115	(1, 1, 0)	-	-	(0, 0, 0)	63974.25	213	(1, 1, 1)
Ratio	**1.0**	**1.0**	**(1.0, 1.0, 1.0)**	**1.13**	**0.57**	**(0.29, 0.82, 0)**	**1.33**	**0.58**	**(0.47, 0.59, 0)**	**2.05**	**0.57**	**(0.12, 0.71, 0)**	**1.04**	**0.94**	**(1.0, 1.0, 1.0)**

Table 3: Interaction Rounds.

	Round 1			Round 2			Round 3	
	NLSW	3-type NSLWs	Feedback	NLSW	3-type NSLWs	Feedback	NLSW	3-type NSLWs
Task3	17	(13, 2, 2)	(0, 0, **1**)	20 (**+3**)	(13, 2, **5**)	-	-	-
Task5	17	(14, 0, 3)	(0, **1**, **1**)	19 (**+2**)	(12, **3**, **4**)	(0, 0, **1**)	21 (**+2**)	(14, 2, **5**)
Task6	9	(6, 3, 0)	(**1**, **1**, **1**)	18 (**+9**)	(**12**, **4**, **2**)	-	-	-
Task7	7	(1, 6, 0)	(**1**, **1**, 0)	14 (**+7**)	(**2**, **12**, 0)	(0, **1**, 0)	15 (**+1**)	(2, **13**, 0)
Task8	17	(10, 4, 3)	(**1**, **1**, **1**)	22 (**+5**)	(**11**, **6**, **5**)	-	-	-
Task10	24	(9, 15, 0)	(**1**, **1**, **1**)	29 (**+5**)	(**10**, **16**, **3**)	(0, **1**, 0)	32 (**+3**)	(12, **18**, 2)
Task13	47	(30, 10, 7)	(**1**, **1**, **1**)	52 (**+5**)	(30, **12**, **10**)	-	-	-
Task14	35	(20, 10, 5)	(**1**, **1**, **1**)	41 (**+6**)	(**24**, 10, **7**)	-	-	-
Task15	39	(11, 21, 7)	(**1**, **1**, **1**)	43 (**+4**)	(**13**, **23**, 7)	-	-	-

indicates compliance). MaskPlace, ChipFormer, and MILP primarily address the non-overlapping constraint, resulting in layouts that may exceed boundaries or lack sufficient spacing. The average results across these non-overlapping layouts are then computed.

Our method consistently produces legal layouts for all benchmarks, with 18% more non-overlapping results than MaskPlace, 41% more than ChipFormer, and 29% more than MILP. We achieve approximately 1.04 HPWL and 0.94 NSLW on average, surpassing a 90% quality level both on HPWL and NSLW compared to manual layouts. Additionally, our method outperforms other baseline methods, with MaskPlace, ChipFormer, and MILP achieving $(1.13, 1.33, 2.05)$ HPWL and $(0.57, 0.58, 0.57)$ NSLW, respectively. MaskPlace and ChipFormer may achieve lower HPWL results than PCBAgent in Task5 due to overlooking spacing and NSWL constraints. However, with the inclusion of refined masks for soft constraints, our method excels in larger-scale tasks like Task16 and 17.

6.3 Effectiveness of LLM-based Interaction

We further evaluate the effectiveness of interaction on different types of NSLW improvement. In PCB design, inductors and capacitors are vital components, with their placement and routing significantly impacting circuit quality. SLWs related to these components are more critical than those of standard components like resistors. To emphasize this importance, we classify surface layer wires into three types: 1) between core components or between core components and chips (SLW type-I), 2) between ordinary components or between ordinary components and chips (SLW type-II), and 3) between core components and ordinary components (SLW type-III). Optimizing these wires presents a real trade-off challenge in the industry.

Table 3 displays the results for the three types of surface layer wires (SLWs) using our LLM-based interaction. The NSLW column indicates the total number of surface line wires, while the 3-type NSLWs column shows the count of SLWs for types I, II, and III. The Feedback column signifies whether a specific SLW type requires

Table 4: Results varying Placement Sequences.

	Net count-oriented		Area-oriented		Ours (Chip-oriented)	
	HPWL	NSLW	HPWL	NSLW	HPWL	NSLW
Task1	456.54	13	237.69	12	439.69	12
Task2	388.65	13	387.62	12	331.33	15
Task3	1352.14	13	1303.61	14	1138.34	20
Task4	1275.71	16	1361.91	18	1257.96	17
Task5	1526.16	18	1008.79	17	1472.74	19
Task6	3037.31	19	2668.06	16	2207.97	21
Task7	2561.99	8	2561.99	7	4835.38	11
Task8	3124.88	21	2963.81	21	2424.28	26
Task9	5603.26	20	6182.9	20	5885.84	23
Task10	2152.28	23	1742.61	23	1204.6	32
Task11	2340.23	25	2551.53	28	2443.96	29
Task12	1789.98	18	1782.63	19	1973.74	39
Task13	5860.62	51	5384.09	51	4590.76	58
Task14	9188.2	37	8935.28	37	6908.99	46
Task15	9423.7	40	9267.38	40	5664.45	46
Task16	51046.23	173	53313.6	172	31792.29	207
Task17	119793.23	158	124964.7	146	63974.25	213
Average	**1.30**	**0.81**	**1.22**	**0.81**	**1.0**	**1.0**

enhancement, with 1 indicating the need for improvement. Our PCBAgent effectively responds to enhancement instructions for one or multiple SLW types across the nine tasks in the table, improving the count of those SLW types accordingly. On average, after 2 or 3 rounds, PCBAgent enhances NSLW by 39.6%, aligning with human experts' preferences for the distribution of the three SLW types.

6.4 Effectiveness of Sequence Optimization

To assess the effectiveness of our sequence decisions, we compare the layout results with two alternative sequencing approaches: 1) a net count-oriented sequence prioritizing components with more connected nets, similar to ChipFormer; 2) an area-oriented sequence prioritizing components with larger areas. Table 4 displays the comparison results of the three sequences. On average, our proposed sequence achieves the lowest HPWL, with reductions of 30% and 22% compared to the net count-oriented and area-oriented methods, respectively. Additionally, our sequence attains the highest NSWL, showing a 19% improvement over the other two methods.

7 CONCLUSION

In this paper, we propose PCBAgent, an agent-based framework for PCB placement that generates satisfactory layouts for high-density tasks, achieving 90% of the quality of manual results in both HPWL and NSLW metrics. Our approach outperforms state-of-the-art methods, delivering the highest quality results.

REFERENCES

[1] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, et al. A graph placement methodology for fast chip design. *Nature*, 594(7862):207–212, 2021.

[2] Ruoyu Cheng and Junchi Yan. On joint learning for solving placement and routing in chip design. *Advances in Neural Information Processing Systems*, 34:16508–16519, 2021.

[3] Ruoyu Cheng, Xianglong Lyu, Yang Li, Junjie Ye, Jianye Hao, and Junchi Yan. The policy-gradient placement and generative routing neural networks for chip design. *Advances in Neural Information Processing Systems*, 35:26350–26362, 2022.

[4] Yao Lai, Yao Mu, and Ping Luo. Maskplace: Fast chip placement via reinforced visual representation learning. *Advances in Neural Information Processing Systems*, 35:24019–24030, 2022.

[5] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. Chipformer: Transferable chip placement via offline decision transformer. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 18346–18364. PMLR, 2023.

[6] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018.

[7] Melanie Mitchell. *An Introduction to Genetic Algorithms*. MIT Press, Cambridge, MA, USA, 1998.

[8] Mohammad Reza Bonyadi and Zbigniew Michalewicz. Particle swarm optimization for single objective continuous space problems: A review. *Evolutionary Computation*, 25(1):1–54, 2017.

[9] Sakait Jain and Hae Chang Gea. Pcb layout design using a genetic algorithm. *Journal of Electronic Packaging, Transactions of the ASME*, 118(1):11–15, March 1996.

[10] Fatimah Sham Ismail, Rubiyah Yusof, and Marzuki Khalid. Optimization of electronics component placement design on pcb using self organizing genetic algorithm (soga). *Journal of intelligent Manufacturing*, 23:883–895, 2012.

[11] Tessy Badriyah, Fitri Setyorini, and Niyoko Yuliawan. The implementation of genetic algorithm and routing lee for pcb design optimization. In *2016 International Conference on Informatics and Computing (ICIC)*, pages 148–153, 2016.

[12] Alex Alexandridis, Evangelos Paizis, Eva Chondrodima, and Marios Stogiannos. A particle swarm optimization approach in printed circuit board thermal design. *Integr. Comput.-Aided Eng.*, 24(2):143–155, jan 2017.

[13] Chung-Kuan Cheng, Chia-Tung Ho, and Chester Holtz. Net separation-oriented printed circuit board placement via margin maximization. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 288–293. IEEE, 2022.

[14] Tung-Chieh Chen, Zhe-Wei Jiang, Tien-Chang Hsu, Hsin-Chen Chen, and Yao-Wen Chang. A high-quality mixed-size analytical placer considering preplaced blocks and density constraints. In *Proceedings of the 2006 IEEE/ACM International Conference on Computer-Aided Design*, pages 187–192, 2006.

[15] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.

[16] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.

[17] Ting-Chou Lin, Devon Merrill, Yen-Yi Wu, Chester Holtz, and Chung-Kuan Cheng. A unified printed circuit board routing algorithm with complicated constraints and differential pairs. In *Proceedings of the 26th Asia and South Pacific Design Automation Conference*, pages 170–175, 2021.

[18] Julia Funke, Stefan Hougardy, and Jan Schneider. An exact algorithm for wirelength optimal placements in vlsi design. *Integration*, 52:355–366, 2016.